Aula 05 — Condições encadeadas

Usando elif, condições compostas e tomadas de decisão com mais de duas possibilidades

Dieimes Nunes

Retomando a Aula 04

Na aula anterior, começamos a criar programas que tomam decisões.

Trabalhamos com:

- `if`;
- `else`;
- comparação de números;
- comparação de textos;
- operador `%`;
- operadores lógicos `and` e `or`;
- teste de mesa com decisões.

Exemplo:

```
if media >= 7:
    print("Aprovado")
else:
    print("Reprovado")
```

O problema da aula

Imagine um sistema de notas com três situações:

- média maior ou igual a 7: aprovado;
- média maior ou igual a 5 e menor que 7: recuperação;
- média menor que 5: reprovado.

Com apenas `if` e `else`, fica mais difícil representar essas três opções de forma clara.

Para isso, usamos `elif`.

Decisões com mais de dois caminhos

No cotidiano, nem toda decisão tem apenas duas opções.

Exemplo:

```
Se a nota for maior ou igual a 7:
    aprovado
Senão, se a nota for maior ou igual a 5:
    recuperação
Senão:
    reprovado
```

Em Python, esse “senão, se” é representado por `elif`.

Atividade 1 — Classificação no cotidiano

No caderno, crie uma regra com pelo menos três possibilidades.

Exemplos:

- classificação de idade;
- nível de bateria;
- qualidade de atendimento;
- prioridade de chamado técnico;
- valor de desconto em uma loja.

Modelo:

```
Se condição 1:
    resultado 1
Senão, se condição 2:
    resultado 2
Senão:
    resultado 3
```

Estrutura com elif

Exemplo em Python:

```
media = float(input("Digite a média: "))

if media >= 7:
    print("Aprovado")
elif media >= 5:
    print("Recuperação")
else:
    print("Reprovado")
```

O Python testa as condições de cima para baixo.

Quando encontra uma condição verdadeira, executa aquele bloco e ignora os demais.

Entendendo a ordem das condições

Se a média for 8, o programa testa:

```
media >= 7
```

Como é verdadeiro, mostra:

```
Aprovado
```

E não testa o restante.

Se a média for 6, o primeiro teste é falso, mas o segundo é verdadeiro:

```
media >= 5
```

Então mostra:

```
Recuperação
```

A ordem pode mudar o resultado

Veja este código:

```
media = 8

if media >= 5:
    print("Recuperação")
elif media >= 7:
    print("Aprovado")
else:
    print("Reprovado")
```

O resultado será:

```
Recuperação
```

Mesmo com média 8.

Isso acontece porque `media >= 5` já é verdadeiro.

Atividade 2 — Teste de mesa com elif

Faça o teste de mesa para três médias:

```
media = 6

if media >= 7:
    situacao = "Aprovado"
elif media >= 5:
    situacao = "Recuperação"
else:
    situacao = "Reprovado"
```

Teste no caderno com:

- média 8;
- média 6;
- média 4.

Para cada caso, informe:

- primeira condição;
- segunda condição;
- situação final.

Atividade 3 — Sistema de situação do aluno

Crie um arquivo chamado:

`situacao_aluno.py`

O programa deve:

1. pedir o nome do aluno;
2. pedir duas notas;
3. calcular a média;
4. classificar a situação:
    - média maior ou igual a 7: aprovado;
    - média maior ou igual a 5: recuperação;
    - média menor que 5: reprovado.

Mostre o nome, a média e a situação.

Classificação por faixa etária

Outro exemplo comum é a classificação de idade.

```
idade = int(input("Digite a idade: "))

if idade < 12:
    print("Criança")
elif idade < 18:
    print("Adolescente")
elif idade < 60:
    print("Adulto")
else:
    print("Idoso")
```

Observe que as condições também são testadas de cima para baixo.

Se `idade` for 10, o primeiro teste já é verdadeiro.

Pensando nas faixas

No exemplo anterior:

```
if idade < 12:
```

pega idades menores que 12.

Depois:

```
elif idade < 18:
```

pega idades de 12 até 17, porque as menores que 12 já foram tratadas no primeiro `if`.

Depois:

```
elif idade < 60:
```

pega idades de 18 até 59.

Atividade 4 — Faixa etária

Crie um arquivo chamado:

`faixa_etaria.py`

O programa deve solicitar uma idade e classificar:

- menor que 12: criança;
- de 12 até 17: adolescente;
- de 18 até 59: adulto;
- 60 ou mais: idoso.

Teste com:

```
10
15
```

```
25
60
```

Depois explique no caderno por que o `else` representa 60 ou mais.

Condições compostas com and

Às vezes, queremos deixar a faixa explícita.

Exemplo:

```
idade = int(input("Digite a idade: "))

if idade >= 12 and idade <= 17:
    print("Adolescente")
else:
    print("Não está na faixa adolescente")
```

O and exige que as duas condições sejam verdadeiras:

- idade maior ou igual a 12;
- idade menor ou igual a 17.

Condições compostas com or

O `or` é usado quando basta uma condição verdadeira.

Exemplo:

```
dia = input("Digite o dia da semana: ")

if dia == "sábado" or dia == "domingo":
    print("Final de semana")
else:
    print("Dia útil")
```

Se o dia for sábado ou domingo, o resultado será final de semana.

Atividade 5 — Dia da semana

Crie um arquivo chamado:

`dia_semana.py`

O programa deve:

1. solicitar o dia da semana;
2. informar se é final de semana ou dia útil.

Regra:

- sábado ou domingo: final de semana;
- qualquer outro dia: dia útil.

Teste com:

```
sábado
domingo
segunda
Sábado
```

Depois observe o que acontece quando a primeira letra é maiúscula.

Melhorando entrada de texto com lower()

Para reduzir problemas com maiúsculas e minúsculas, podemos usar `.lower()`.

```
dia = input("Digite o dia da semana: ").lower()

if dia == "sábado" or dia == "domingo":
    print("Final de semana")
else:
    print("Dia útil")
```

O método `.lower()` transforma o texto digitado em letras minúsculas.

Exemplo:

```
"Sábado".lower()
```

Condições aninhadas

Condição aninhada é uma condição dentro de outra condição.

Exemplo:

```
usuario = input("Usuário: ")
senha = input("Senha: ")

if usuario == "admin":
    if senha == "1234":
        print("Acesso permitido")
    else:
        print("Senha incorreta")
else:
    print("Usuário não encontrado")
```

Esse tipo de estrutura pode ser útil, mas precisa de cuidado para não deixar o código confuso.

Comparando com and

O exemplo anterior também poderia ser escrito assim:

```
usuario = input("Usuário: ")
senha = input("Senha: ")

if usuario == "admin" and senha == "1234":
    print("Acesso permitido")
else:
    print("Acesso negado")
```

Essa versão é menor.

Mas ela não informa se o problema foi o usuário ou a senha.

A escolha depende da necessidade do programa.

Atividade 6 — Login simples

Crie um arquivo chamado:

`login.py`

O programa deve solicitar:

- usuário;
- senha.

Regras:

- usuário correto: `admin`;
- senha correta: `1234`.

Faça de duas formas:

1. usando `and`;
2. usando condição aninhada para mostrar mensagens diferentes.

Sistema de desconto por valor

Agora vamos aplicar `elif` em uma situação de loja.

Regra:

- compra acima de R$ 500,00: 15% de desconto;
- compra acima de R$ 200,00: 10% de desconto;
- compra acima de R$ 100,00: 5% de desconto;
- demais compras: sem desconto.

A ordem das condições importa.

Devemos começar pelo maior valor.

Programa: desconto progressivo

```
total = float(input("Digite o total da compra: "))

if total > 500:
    desconto = total * 15 / 100
elif total > 200:
    desconto = total * 10 / 100
elif total > 100:
    desconto = total * 5 / 100
else:
    desconto = 0

valor_final = total - desconto

print(f"Desconto: R$ {desconto:.2f}")
print(f"Valor final: R$ {valor_final:.2f}")
```

Atividade 7 — Teste de mesa do desconto

Faça o teste de mesa para os valores:

- R$ 80,00;
- R$ 150,00;
- R$ 300,00;
- R$ 600,00.

Para cada valor, informe:

Total	Percentual	Desconto	Valor final
80			
150			
300			
600			

Depois execute o programa para conferir.

Atividade 8 — Caixa com desconto progressivo

Crie um arquivo chamado:

`caixa_desconto_progressivo.py`

O programa deve solicitar:

- nome do cliente;
- nome do produto;
- preço unitário;
- quantidade.

Depois deve calcular o subtotal e aplicar a regra:

- acima de R$ 500,00: 15%;
- acima de R$ 200,00: 10%;
- acima de R$ 100,00: 5%;
- até R$ 100,00: sem desconto.

Mostre o resumo da compra.

Tratando opções de menu

Muitos sistemas usam menus.

Exemplo:

```
1 - Cadastrar
2 - Consultar
3 - Alterar
4 - Sair
```

Com `if`, `elif` e `else`, conseguimos tratar a opção escolhida.

```python
opcao = int(input("Digite a opção: "))

if opcao == 1:
    print("Cadastrar")
elif opcao == 2:
    print("Consultar")
elif opcao == 3:
    print("Alterar")
elif opcao == 4:
    print("Sair")
```

```
else:
    print("Opção inválida")
```

Atividade 9 — Menu simples

Crie um arquivo chamado:

`menu.py`

Mostre o menu:

```
1 - Somar dois números
2 - Subtrair dois números
3 - Multiplicar dois números
4 - Dividir dois números
```

O usuário escolhe uma opção e informa dois números.

O programa deve mostrar o resultado da operação escolhida.

Se a opção for inválida, mostrar mensagem de erro.

Cuidado com divisão por zero

Na opção de divisão, precisamos evitar erro.

```
if numero2 == 0:
    print("Não é possível dividir por zero")
else:
    resultado = numero1 / numero2
    print(resultado)
```

Esse é um exemplo de validação.

Validação é verificar se os dados fazem sentido antes de executar uma operação.

Erros comuns nesta aula

Erro na ordem das condições:

```
if total > 100:
    desconto = 5
elif total > 500:
    desconto = 15
```

O valor acima de 500 nunca chegará na segunda condição.

Erro de lógica com `and`:

```
if idade >= 12 or idade <= 17:
    print("Adolescente")
```

Essa condição não representa corretamente a faixa de 12 a 17.

O correto é usar `and`.

Teste de mesa e qualidade

Antes de entregar um programa, teste casos diferentes.

Para o sistema de notas, teste:

- média 10;
- média 7;
- média 6.9;
- média 5;
- média 4.9;
- média 0.

Esses valores ajudam a encontrar erros nos limites das condições.

Os limites são pontos em que muitos erros acontecem.

Desafio final da aula

Crie um programa chamado:

`sistema_avaliacao.py`

O programa deve solicitar:

- nome do aluno;
- três notas;
- frequência.

Regras:

- média maior ou igual a 7 e frequência maior ou igual a 75: aprovado;
- média maior ou igual a 5 e frequência maior ou igual a 75: recuperação;
- frequência menor que 75: reprovado por frequência;
- demais casos: reprovado por nota.

Mostre:

Fechamento

Hoje trabalhamos com decisões mais completas.

Vimos:

- `elif`;
- condições encadeadas;
- condições compostas com `and` e `or`;
- condições aninhadas;
- classificação por faixa;
- descontos progressivos;
- menus;
- validação de divisão por zero;
- testes com valores de limite.

Na próxima etapa do curso, vamos iniciar as estruturas de repetição.